

NYC Bike Share Pipeline & Machine Learning Project

This document serves as the comprehensive, highly detailed blueprint for recreating and understanding the End-to-End Data Engineering and Machine Learning project.

It is designed not just to give you the commands, but to explain the "Why" behind every architectural decision, making it perfect for studying or walking a recruiter through your portfolio.

Master Architecture Overview

This project abandons traditional local installations in favor of a modern **Microservices Architecture** using Docker. Instead of installing PostgreSQL, Airflow, and Python directly onto your Windows/Mac host OS, we spin up 5 isolated, specialized "mini-computers" (containers) that communicate over a private Docker network.

The 5 Microservices:

1. **PostgreSQL (bike_share_db)**: The central Data Warehouse. It holds all raw and cleaned data.
2. **pgAdmin (pgadmin_ui)**: A web-based graphical interface to interact with the database without writing terminal commands.
3. **Apache Airflow (airflow_standalone)**: The Orchestrator. It watches the clock and automatically triggers data extraction, transformation scripts, and Machine Learning retraining.
4. **FastAPI (fastapi_server)**: The Machine Learning Backend. It loads the AI model into memory and serves predictions via a REST API.
5. **Streamlit (streamlit_ui)**: The Frontend User Interface. A "Pro" responsive web app featuring 3D PyDeck maps and interactive charts for AI predictions.

Expected Directory Structure

Before starting, your main project folder (e.g., `bike_share_pipeline`) should look exactly like this:

```
bike_share_pipeline/
├── dags/
│   ├── bike_share_dag.py      # Airflow ETL instructions (5-min schedule)
│   └── model_training_dag.py  # Automates ML training (15-min schedule)
├── scripts/
│   ├── ingest.py             # Pulls API data (Extract & Load)
│   └── transform.py           # Cleans data via SQL (Transform)
├── init.sql                   # Auto-creates database tables on first boot
├── docker-compose.yml         # The master blueprint linking all 5 containers
├── train_model.py             # The ML script that Airflow runs internally
├── app.py                     # FastAPI server code
├── frontend.py                # Streamlit UI code (with PyDeck maps)
├── requirements.txt           # Python dependencies for Airflow (Includes ML
└── bike_model.pkl             # Generated AI brain (created after Phase 4)
```

└─ .gitignore

Instructs Git to ignore virtual environments

Phase 1: Local Environment Setup

Before touching Docker, we needed a safe place to test Python scripts locally.

1. **Create the Environment:** We used Python's built-in `venv` module to create an isolated sandbox so our project dependencies wouldn't conflict with other projects on your PC.
2. **Activation:**
 - * Windows: `venv\Scripts\activate`
 - Mac/Linux: `source venv/bin/activate`
3. **Core Dependencies Installed:**
 - `pandas` , `numpy` (Data manipulation)
 - `scikit-learn` , `joblib` (Machine Learning algorithms)
 - `sqlalchemy` , `psycopg2-binary` (PostgreSQL database connectors)
 - `fastapi` , `uvicorn` , `streamlit` , `pydeck` (Web and UI frameworks)

Phase 2: The Database Layer (Storage)

Every data project needs a robust storage layer. We chose **PostgreSQL 15**.

1. **The Schema (`init.sql`):**
 - We created `raw_station_status` to dump the exact, unaltered JSON responses from the Citi Bike API.
 - We created `ml_feature_table` for the polished, mathematically ready data (e.g., breaking timestamps down into `hour_of_day` and `is_weekend`).
2. **Persistent Storage (Volumes):** In `docker-compose.yml` , we mapped a volume (`pgdata_new:/var/lib/postgresql/data`). This ensures that if the Docker container crashes or is deleted, the actual hard drive data survives on your host machine.
3. **Port Mapping:** We mapped internal port `5432` to external port `5434` so our local Python scripts could talk to the database from the outside.

Phase 3: The ETL Pipeline (Apache Airflow)

To make the AI smart, we needed a massive amount of historical data. We deployed **Apache Airflow** for orchestrating a **Near Real-Time** pipeline.

1. **Extract & Load (`ingest.py`):** Connects to the live NYC Citi Bike API, downloads the current status of hundreds of stations, and inserts the raw rows into Postgres. It features per-row error handling to ensure a single bad API record doesn't fail the batch.
2. **Transform (`transform.py`):** Runs immediately after ingestion. It uses an `etl_watermark` table to guarantee rows are only processed once, and utilizes a `LATERAL JOIN` to perfectly match station readings to the closest point-in-time weather data.
3. **The DAG (Directed Acyclic Graph):** We wrote `bike_share_dag.py` to tell Airflow to run Task 1(`ingest`), then run Task 2(`transform`) on a **5-minute schedule** (`schedule_interval='*/5 * * * *`).
4. **Airflow Initialization:** Because we run Airflow in `standalone` mode, it dynamically generates its own admin password on the first boot. We explicitly override this by executing a user-creation script via the Docker CLI to ensure standardized access.



Phase 4: Machine Learning (Continuous Training / MLOps)

We transitioned from manual script execution to a fully automated Continuous Training (CT) pipeline.

1. **Automated Orchestration:** We created `model_training_dag.py` to run every 15 minutes. By mounting the root directory into the Airflow container, Airflow can automatically execute `train_model.py`.
2. **The Model Bake-Off:** When triggered, `train_model.py` queries the internal database (`postgres:5432`), pulls the latest cleaned data, and instantiates four different models: *Linear Regression*, *Decision Tree*, *Random Forest*, and *Gradient Boosting*.
3. **Training & Scoring:** The script splits the data (80% to learn, 20% to test) and grades each model using **Mean Absolute Error (MAE)**.
4. **Exporting:** The script automatically takes the winning model and serializes it into `bike_model.pkl` using the `joblib` library, saving it directly back to the project root directory.



Phase 5: The Live Web Application

1. **The API / Backend (`app.py`):** FastAPI loads the AI brain (`bike_model.pkl`) into memory upon startup and exposes a `/predict` endpoint that returns AI forecasts.
2. **The UI / Frontend (`frontend.py`):** We built a "Pro" Streamlit dashboard. It features an interactive **PyDeck 3D map** of NYC, dynamic historical trend charts, and dropdowns populated by live API station data (with built-in request timeouts for stability).
3. **Internal Docker Networking:** Streamlit talks directly to FastAPI using its internal container name (`http://api:8000/predict`), entirely bypassing `localhost`.



Phase 6: Version Control & GitHub Deployment

To showcase this project in your portfolio, it's maintained on GitHub using Git version control.

1. **.gitignore Setup:** The project uses a `.gitignore` file to ensure virtual environments (`venv/`), Python cache files (`__pycache__/`), and local `.env` files are not uploaded to the public repository. Note: `bike_model.pkl` is intentionally tracked so the frontend has a pre-trained model immediately upon cloning.
2. **Pushing Code:** The code is uploaded via the standard Git workflow:

```
git init
git config --global user.email "your.email@example.com"
git config --global user.name "Your Name"
git add .
git commit -m "Initial commit: End-to-End MLOps Bike Share Pipeline"
git branch -M main
git remote add origin [https://github.com/YOUR_USERNAME/YOUR_REPO_NAME.git]
```

```
(https://github.com/YOUR_USERNAME/YOUR_REPO_NAME.git)
git push -u origin main
```



Command Reference Cheat Sheet

Docker Operations

- **Start all containers:** `docker-compose up -d`
- **Stop all containers:** `docker-compose down`
- **Reload AI Model in Web App:** `docker-compose restart api`
- **Rebuild specific container:** `docker-compose up -d --build frontend`

Database Interaction (CLI)

- **Open PSQL inside the container:** `docker exec -it bike_share_db psql -U admin -d bikeshare`
- *Inside PSQL:*
 - List tables: `\dt`
 - Row count check: `SELECT COUNT(*) FROM raw_station_status;`
 - Check High-Water Mark: `SELECT * FROM etl_watermark;`
 - Exit: `\q`



How to Activate and Run the Project

Whenever you deploy this project fresh or restart your computer:

1. **Start Docker Engine:** Open Docker Desktop and wait for the green icon.
2. **Launch Services:** Open VS Code terminal in the project folder and run:

```
docker-compose up -d
```

3. **Initialize Airflow User (Mandatory first-run step):** Airflow `standalone` ignores default environment variables for user creation. Wait 60 seconds after spinning up the containers, then run this to explicitly set your login:

```
docker exec -it airflow_standalone bash -c "airflow users
```

```
create --username laukik --firstname Laukik --lastname
```

```
User --role Admin --email laukik@example.com --password laukik"
```

4. **Access Control Panels:**

- 🎨 **User Interface (Streamlit):** <http://localhost:8501>
- ⚙️ **Pipeline Manager (Airflow):** <http://localhost:8080> (Login: laukik / laukik)
- 🗄️ **Database UI (pgAdmin):** <http://localhost:5050>

5. **Apply Automated AI Updates:** Airflow now automatically retrains your AI model every 15 minutes in the background. To apply the newest, smartest AI to your live frontend, simply restart the FastAPI container:



```
docker-compose restart api
```

Troubleshooting Guide

- **Error: Airflow "Invalid Login" or "CSRF Token Missing / Bad Request"**
 - *Cause:* Corrupted database volume or stale browser cookies.
 - *Fix:* Run `docker-compose down`, run `docker volume rm bike_share_pipeline_airflow_db`, and run `docker-compose up -d`. Clear your browser cache or use Incognito mode. Re-run the `docker exec` user creation command from Step 3.
- **Error: role "postgres" does not exist when using psql**
 - *Cause:* Running the `psql` command on your Windows host instead of inside the Docker container.
 - *Fix:* Always use `docker exec -it bike_share_db psql ...` to run database commands.
- **Error: Streamlit UI predicts the same number (e.g., 14.0) every time**
 - *Cause:* The AI model is under-fit due to a lack of data variance (Airflow hasn't collected enough days of data yet).
 - *Fix:* Let the Airflow pipeline run for at least 24-48 hours. The background training DAG will automatically pick up the new variance. Then run `docker-compose restart api` to load the improved model.